# 포트폴리오

AI 검증 체계, 성능, 장애 격리, 다중 노드 정합성, 외부 인증 연동 사례 5건과 직접 만든 도구 3개

Java·Spring 백엔드 5년 1개월(2021.10~2026.09, (주)싸이웍스). 사례마다 먼저 적은 것은 제가 그때 겪은 대로 쓴 말이고, 그 아래는 한 일과 실측값입니다. 회사 자료(코드·화면·설계 문서)는 싣지 않았습니다. 연락처는 이력서에 있습니다. 2026.11

---

**1. 문제 해결 사례** (AI 검증 → 성능·DB 설계 → 운영 장애 → 데이터 정합성 → 외부 연동)

1-1. AI 코딩 에이전트 산출물 검증 체계 — 작업 명세·커밋 전 게이트·교차 리뷰 (자사 CMS, 2026)

1-2. 학사 DB 원격 조회를 야간 적재로 전환 — 운영 DB 실측·인덱스 설계 (건양대, 2025~2026)

1-3. DB 연결 장애 중 메인 화면 응답 유지 — 결과 캐시 + DB 호출 차단기 (CETIS, 2025~2026)

1-4. 다중 노드 야간 배치 중복 실행 3중 차단 — DB 선점 행·멱등 upsert (CETIS, 2026)

1-5. 운영 중인 로그인을 멈추지 않고 KG이니시스 통합인증 추가 — 외부 인증 병행 연동 (CETIS, 2026)

**2. 개인 프로젝트** (업무 외 · Electron·React·TypeScript, Python · 요구사항·구조·검증은 직접, 구현은 AI 코딩 에이전트와)

2-1. BLACKPINK — 로컬 개발환경 관제 앱 (Electron·React·TypeScript, 공개)

2-2. QA-A — 비개발 QA 담당자용 화면 주석 → Jira 등록 도구 (Python·CDP, 공개)

2-3. JOB_YK — 채용 공고 수집·채점 도구 (Electron·React·TypeScript)

회사 자료(코드·화면·설계 문서)는 싣지 않았습니다. 수치는 운영 DB와 로그에서 직접 잰 값이고, 그림은 설명을 위해 새로 그렸습니다. 2026.10

## 사례 1-1 AI 코딩 에이전트 산출물 검증 체계

**자사 CMS 솔루션 신규 개발** · 2026.04 ~ 2026.09 · 개발 3명 + 기획·디자인·퍼블리셔 · 기여(커밋 기준): 저장소 전체 커밋 2,219건 중 본인 1,564건(70%) · Java 8, Spring Boot 2.7, Thymeleaf, PostgreSQL, Playwright, Claude Code, Codex CLI

**11종**

> 계획을 세우고 시작하고, 진행 전후에는 꼭 자가검증 혹은 비판적으로 보는 부분을 진행하게 합니다.

커밋 전 자동 게이트 (정적·실행 요구·문서 검사 3층)

**85건**

Playwright 회귀 스위트 통과 유지

**186건**

이 흐름으로 처리한 CMS 티켓 (2026.06~09)

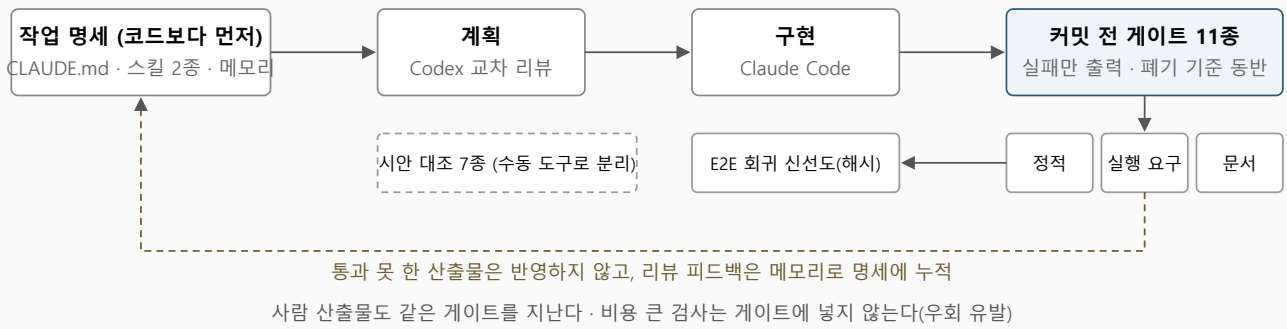
**2건**

권한별 화면 전수 자동 검증으로 찾은 실제 결함

Claude Code에 구현을 맡기자 검수가 사람 눈에 의존했고, "안 읽은 파일을 결함으로 보고"하는 과대보고가 섞여 믿을 수 있는 범위가 불분명했다. 같은 화면 QA가 나흘 연속 반복되기도 했다. 맡기는 것은 읽기·분석·초안·반복 구현. 검증은 사람 산출물과 같은 문턱을 자동으로 통과하게 하고, 믿지 못하는 부분은 다른 모델이 다시 본다. 에이전트가 지킬 기준은 코드보다 먼저 정한다.

CLAUDE.md 공통 명세·스킬 2종·정본 문서를 먼저 정하고 피드백은 메모리로 누적. 커밋 전 훅에 자동 게이트 11종을 세 층(정적 검사·실행 요구 검사·문서 검사)으로 두고 실패만 출력, 게이트마다 폐기 기준 동반. Playwright E2E 회귀 spec의 신선도(내용 해시)를 게이트로 강제. 실행 비용이 큰 시안 대조 7종은 우회를 유발할 수 있어 수동 도구로 분리. 계획 단계부터 Codex로 교차 리뷰하고 "못 읽은 것을 결함으로 보고하지 않는다" 등 과대보고 방지 조항을 리뷰 기준에 명시.

반복 QA를 커밋 단계에서 차단, Playwright 회귀 85건 통과 유지, 권한별 화면 전수 자동 검증으로 실제 결함 2건 수정. 2026.06~09 CMS 티켓 186건을 이 흐름으로 처리했고, 명세·스킬을 팀 표준으로 문서화했고, 감시 모델이 수행 모델의 결과를 검증하는 이 방식을 실무 과제 8건에 적용한 내용을 2026.07 사내 스터디에서 발표했다. CETIS 통합인증 반영 전 교차 리뷰에서는 계정 탈취 가능 결함 포함 4건을 사전에 수정했고, AI가 보고한 N+1은 실행계획 실측으로 오탐 판정했다.




검증이 사람 눈에서 커밋 전 자동 게이트로 옮겨 갔다. 티켓 건수나 리드타임이 달라졌다고는 말하지 않는다.

## 사례 1-2 학사 DB 원격 조회를 야간 적재로 전환

**건양대 학생역량 포트폴리오 시스템** · 2025.05 ~ 2026.09 · 개발 단독 · 기여(커밋 기준): 저장소 전체 커밋 698건 중 본인 629건 (90%) · Java 8, Spring 4, MyBatis, PostgreSQL 15, Oracle 학사 DB 연동(oracle_fdw)

**129초 → 31ms**

> 실제로 속도가 빨라졌다는 얘기를 들었습니다.

이수정보 전건 집계 조회

**35ms → 1ms**

조인 조회

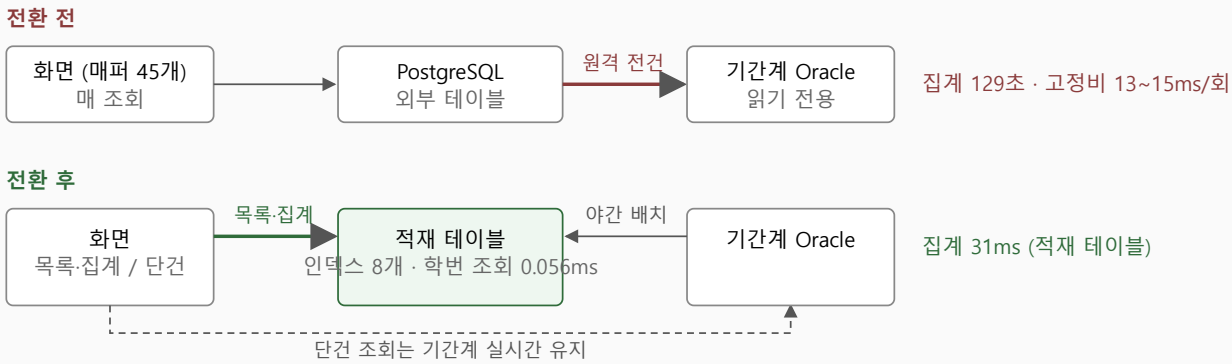
**19.99ms → 0.056ms**

학번 단건 조회 (인덱스 8개)

학사 정보는 학교 기간계 Oracle(읽기 전용)에 있고, 화면 매퍼 45개가 PostgreSQL 외부 테이블로 매 조회마다 원격을 읽어 집계·목록이 분 단위였다. 조건에 따라 원격 전건(124초)을 가져오는 경우가 섞여 어느 화면이 왜 느린지 설명이 사람마다 달랐다. 추측하지 않고 운영 DB에서 실측했다. 등치·IN·LIKE는 원격으로 내려가지만 문자열 범위 조건과 NVL은 내려가지 않아 전건을 가져온다는 것, 외부 조회 1회당 13~15ms 고정비가 있어 400~1,000행이 손익분기라는 것을 확인했다. 전부 적재하면 적재 시간이 늘고 성적 정정 반영이 늦어지므로 목록·집계용 뷰만 야간 적재하고 단건 조회는 실시간으로 남기기로 했다.

기간계 뷰를 야간 배치로 적재 테이블에 옮기고, 적재 범위는 당해+전년도로 학교 담당자와 확정했다. 인덱스 8개를 설계하며 적재 시간 영향을 함께 점검하고, JDBC fetchSize를 10에서 1,000으로 조정했으며, 연도·날짜 조건을 IN 목록으로 쓰는 작성 규칙을 문서화했다.

이수정보 전건 집계 129초 → 31ms, 조인 조회 35ms → 1ms, 학번 단건 19.99ms → 0.056ms. 적재 뷰와 실시간 뷰가 공존하는 상태와 실시간 복귀 불가 이유를 실측 표로 인수인계서에 남겼다.




실시간 조회를 전부 없애지 않고 목록·집계만 적재로 옮긴 부분 전환. 실시간 복귀가 불가능한 이유(원격 pushdown 조건·고정비)를 실측 표로 남겼다.

## 사례 1-3 DB 연결 장애 중 메인 화면 응답 유지

**건설기술인교육관리 통합시스템(CETIS)** · 2025.05 ~ 2026.09 · 개발 단독 · 기여(커밋 기준): 저장소 전체 커밋 1,620건 중 본인 1,492건(92%) · Java 8, Spring MVC 4.3, 전자정부 3.10, MyBatis, PostgreSQL

### 40초 대기 → 응답 유지

> 조금 더 빨리 생각해서 처리할 걸 그랬다는 아쉬움이 있었습니다.

DB 연결 장애 중 메인 화면 (전환 전 커넥션 타임아웃 40초)

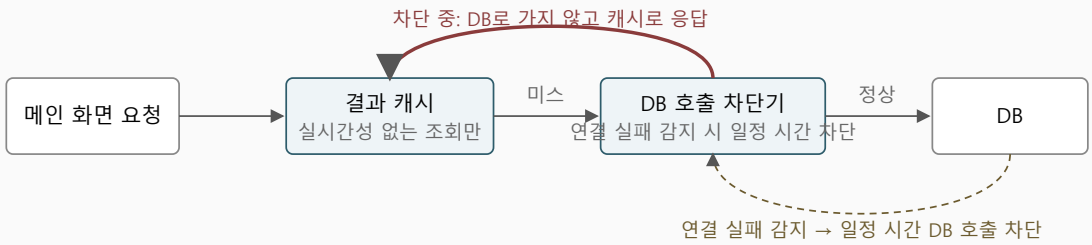
### 52건

운영 로그 사전 분석으로 처리한 티켓 (2026.07~09)

운영 로그를 일·주 단위로 점검하다가 DB 재기동·연결 장애 때 메인 화면이 커넥션 타임아웃까지 40초를 기다리는 것을 먼저 발견했다. 사용자 문의는 없었다. 메인 화면의 조회 대부분은 실시간성이 없는데도 DB가 응답할 때까지 전체가 막혔다. 장애 중에도 실시간성이 없는 조회는 응답할 수 있어야 한다. 캐시만 두면 캐시에 없는 요청은 장애 중에도 DB로 가서 타임아웃을 기다리므로, 연결 실패를 감지하면 일정 시간 DB 호출 자체를 막는 층이 따로 필요하다.

실시간성이 필요 없는 조회에만 결과 캐시를 두고, 연결 실패 감지 시 일정 시간 DB 호출을 막는 차단기(서킷 브레이커)를 직접 구현했다. 차단 중에는 캐시 결과로 응답한다.

DB 연결 장애 중에도 메인 화면이 응답한다. 같은 시스템에서 운영 로그를 일·주 단위로 먼저 분석해 티켓 전에 수정하는 절차를 고정했고, 2026.07~09 티켓 52건을 이 흐름으로 처리했다.




캐시와 차단기를 분리한 이유: 캐시만으로는 캐시에 없는 요청이 장애 중 타임아웃을 기다린다. 차단 중에는 DB 호출 자체가 없다.

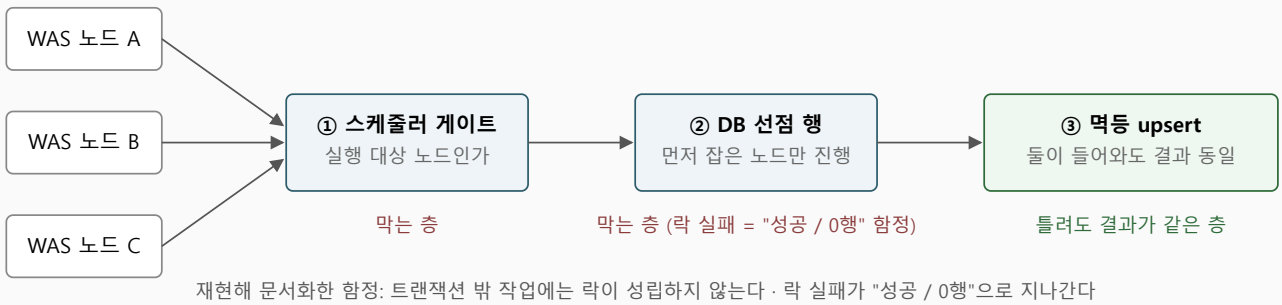
## 사례 1-4 다중 노드 야간 배치 중복 실행 3중 차단

**건설기술인교육관리 통합시스템(CETIS)** · 2026 · 개발 단독 · Java 8, Spring @Scheduled, MyBatis, PostgreSQL

운영 WAS가 여러 대라 야간 집계 배치가 노드마다 따로 돌았고, 오류 없이 결과가 중복 반영됐다. JVM 안의 실행 플래그로는 다른 노드를 막을 수 없다. 락 하나에만 의존하면 락이 실패했을 때 조용히 지나간다. "막는 층"과 "틀려도 결과가 같은 층"을 따로 둔다.

① 스케줄러 게이트(이 노드가 실행 대상인지 판정) → ② DB 선점 행(먼저 잡은 노드만 진행) → ③ 멱등 upsert(둘이 들어와도 결과 동일). 트랜잭션 밖 작업에는 락이 성립하지 않는다는 것, 락 실패가 "성공 / 0행"으로 지나간다는 것을 재현해 문서화했다.

중복 반영 차단. 같은 패턴을 다른 배치에 적용할 수 있게 작성 규칙으로 남겼다. 이 경험이 메시지 큐 멱등 소비자 설계로 이어 가려는 다음 학습 과제다.




재현해 문서화한 함정: 트랜잭션 밖 작업에는 락이 성립하지 않는다 · 락 실패가 "성공 / 0행"으로 지나간다

①②가 모두 뚫려도 ③이 결과를 같게 만든다. 반대로 ③만 있으면 노드마다 같은 일을 중복해서 한다. 노드 A·B·C는 설명용 예시이며 실제 노드 수가 아니다.

## 사례 1-5 운영 중인 로그인을 멈추지 않고 KG이니시스 통합인증 추가 — 외부 인증 병행 연동

**건설기술인교육관리 통합시스템(CETIS)** · 2026 · 개발 단독 · 기여(커밋 기준): 저장소 전체 커밋 1,620건 중 본인 1,492건(92%) · Java 8, Spring MVC 4.3, 전자정부 3.10, MyBatis, PostgreSQL

**0건**

> 가이드를 확인하고 진행하고 실제로 작업 전에도 계획을 먼저 세우고 진행했기 때문에 별도로 그런 일은 없었습니다.

기존 NICE 로그인 코드 수정 건수 (수단 추가 전후)

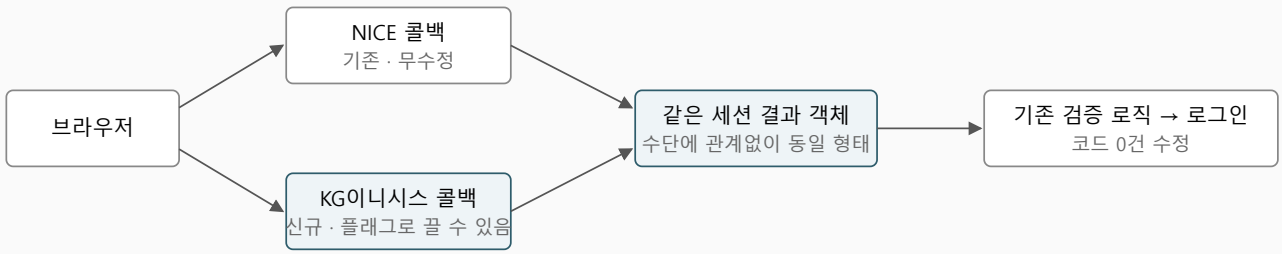
**2종**

운영 중인 본인확인 수단 (NICE → NICE + KG이니시스)

운영 중인 NICE 본인확인 로그인을 멈추지 않고 KG이니시스 통합인증을 하나 더 붙여야 했다. 새 수단이 기존과 같은 세션 결과를 만들면 기존 검증 로직은 그대로 둘 수 있다. 끌 수 있는 스위치가 있어야 반영 뒤 문제가 생겨도 되돌릴 수 있다.

이니시스 콜백이 NICE와 같은 세션 결과 객체를 만들게 해 기존 검증 로직을 재사용하고, 플래그로 이니시스 경로를 끌 수 있게 배포했다. 반영 전 Codex 교차 리뷰에서 계정 탈취가 가능한 결함을 찾아 수정했고, AI가 보고한 N+1은 실행계획을 실측해 오탐으로 판정했다. 반영 전 최다 호출 URL을 감사해 인가 누락을 확인했다.

기존 로그인 코드 0건 수정으로 수단을 추가했다. 붙인 뒤 상대 쪽이 바뀐 연동도 있었다. 오픈배지 API 응답 형식이 바뀌어 깨진 갱신은 상대 응답을 실측해 복구했고(건양대), Daum 주소 API 도메인 폐기 때는 CSP 허용 목록까지 영향 범위를 추적해 40여 기관 사이트를 Kakao CDN으로 일괄 이관했다. 통신사 API·결제 승인/정산 API 연동 경험은 없다.




반영 전: Codex 교차 리뷰 · 최다 호출 URL 인가 감사 · N+1 보고는 실행계획으로 오탐 판정

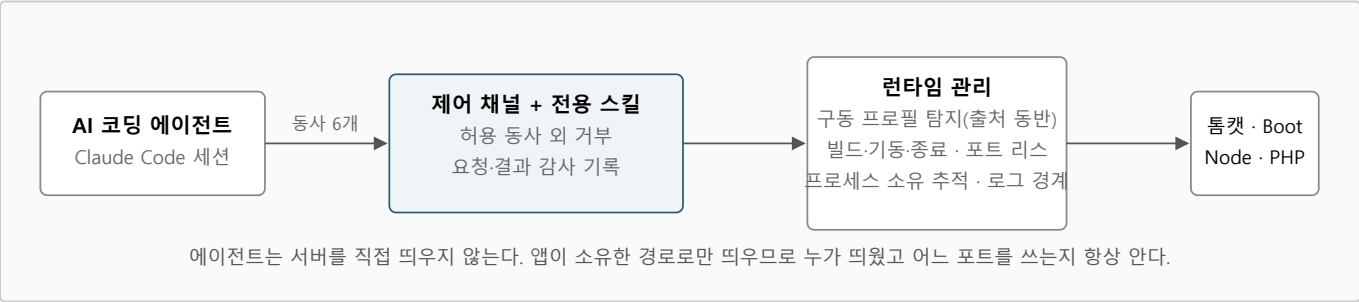
두 콜백이 같은 세션 결과 객체를 만들어 기존 검증 로직에 합류한다. 플래그는 반영 뒤 되돌릴 수단이다.

## 개인 2-1 BLACKPINK — 로컬 개발환경 관제 앱

**2026.08 ~ 2026.09** · 1인 · 요구사항·구조·검증은 직접, 구현에 Claude Code 사용 · Electron, React, TypeScript, Tailwind, Zustand, node-pty · 공개판 **github.com/wkddbsrua/BLACKPINK-PUB**

> 실제로 터미널 창 켜고, 소스가 있는 위치로 이동해서 Claude 등을 명령어로 실행해서 CLI 환경에서 개발하는 것도 매번 하는 게 귀찮게 느껴졌고, 그래서 해당 경로 하위에 있으면 Claude를 바로 실행할 수 있거나, 터미널을 바로 띄울 수 있도록에서 시작하고, 톰캣을 구동하는 데 있어서 레거시 프로젝트들의 경우 거의 비슷한 환경에서의 실행인데 포트만 달라지는 부분에 계속 그런 걸 신경 쓰는 부분이 귀찮아서 만들게 되었습니다

- 구동하기 전에 "그 정답이 어디에 적혀 있는지"부터 찾는다. 값마다 출처 파일과 원문 조각을 붙이고, 확정한 프로필은 원본 지문(size·mtime)과 함께 저장해 원본이 바뀌면 낡았다고 표시한다.
- 유형(외장 톰캣 / Gradle·Spring Boot / Node / PHP)마다 규칙을 다르게 둔다. 모양이 다르면 없는 사실을 만들어 내지 않는다.
- AI 에이전트는 동사 6개로 제한한 제어 채널과 전용 스킬로만 서버를 띄운다. 앱이 프로세스 소유를 추적하므로 포트 리스·로그 경계·종료가 따라온다.
- 앱이 파일을 바꾸거나 지운 기록은 감사 원장에 남기고, 삭제는 격리소를 거친다.
- 본인 일상 도구로 사용. 톰캣·Spring Boot·Node·PHP 프로젝트의 기동·빌드·종료·포트를 한 화면에서 제어.
- Claude Code 세션은 이 앱을 통해서만 서버를 띄우도록 전역 지침과 스킬로 고정.
- Jira 이슈를 Claude 세션으로 넘기는 창구, 프로젝트 메모리·계획 관리, 문서 사용 여부 판정(근거 동반)을 같은 앱 안에 둠.
- Windows 주력, macOS 포팅 진행 중(되는 것과 안 되는 것을 실측 문서로 관리).



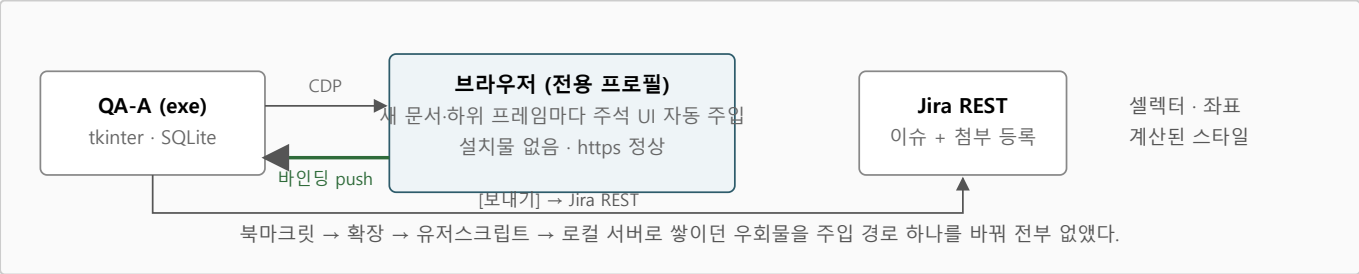

에이전트는 서버를 직접 띄우지 않는다. 앱이 소유한 경로로만 띄우므로 누가 띄웠고 어느 포트를 쓰는지 항상 안다.

## 개인 2-2 QA-A — 비개발 QA 담당자용 화면 주석 → Jira 등록 도구

**2026.08 ~ 2026.09** · 1인 · 구현에 Claude Code 사용 · Python(tkinter), Chrome DevTools Protocol 클라이언트 직접 구현, SQLite, Jira REST · 주석 UI는 agentation 원본을 쓰고 문구만 한글화 · exe 배포 · **github.com/wkddbsrua/QA-A**

> AI가 코딩은 잘하는데 Figma의 디자인에서 잘못된 부분 혹은 실제와 다른 위치 등의 문제가 생겼을 경우에는 제대로 대응하지 못하는 문제가 있는데, 사람이 하더라도 그 부분을 스크린샷 찍어서 다시 안내하고 하는 방향 때문에, QA-A의 경우에는 실제 그 영역을 확인해서 영역 내의 문제를 AI에게 넘겨줄 수 있도록 하는 방향에서 시작했습니다

- 원인은 주입 경로 하나였다. 프로그램이 브라우저를 띄우고 CDP로 주입하면 두 한계가 동시에 사라진다. 브라우저 설치물 0, 페이지당 클릭 0. 직접 구현한 범위는 CDP 클라이언트·주입·프레임 대응·수집·Jira 연동이고, 주석 UI는 agentation 기반이다.
- 모든 새 문서·하위 프레임에 자동 실행, 주석은 네트워크를 타지 않고 바인딩으로 곧장 프로그램으로.
- iframe 셸 구조에서 "떠 있는가"를 window 플래그가 아니라 DOM으로 판단하고, 자가확인은 프레임 전체와 실제 고정 배치 여부까지 본다. 진단은 숫자로 남긴다.
- 모달 위에 올리기 위해 z-index 경쟁을 하지 않는다. 이 도구는 주석만 하고 VRT는 넣지 않는다.
- exe로 배포해 QA 담당자 1명이 검수 5~10회차에 사용.
- 화면 지적과 셀렉터·좌표·계산된 스타일을 수집하고, [보내기]를 누르면 프로그램이 Jira REST로 이슈·첨부를 등록. 개발자는 지적마다 셀렉터·좌표를 받는다.
- 전용 프로필로 열리므로 처음 한 번만 로그인하면 유지.



## 개인 2-3 JOB_YK — 채용 공고 수집·채점 도구

**2026.10** · 1인 · 구현에 Claude Code 사용 · Electron, React, TypeScript

- **채용 사이트는 여러 군데인데 실제로 나한테 맞는 직무·지역을 보는 방법은 사이트마다 다르기 때문에 내가 원하는 내용을 바로바로 수집해서 보고 즐겨찾기 해 두고, 실제 채용하는 데 도움을 받기 위해서였고, 지금도 매일매일 수집하고 사용 중에 있습니다** 채용 사이트마다 공고를 열어 자격요건을 내 경력과 대조하는 일을 매일 반복했고, 어제 본 공고와 오늘 본 공고가 섞였다.
- 공고 본문의 자격요건·주요업무·우대 줄마다 보유 1 · 인접 0.5 · 없음 0으로 체크표를 만들고 섹션 가중(자격 1.0 · 업무 0.7 · 우대 0.5)으로 매치율을 낸다. 매치율과 별개로 회사 신호(성장·규모·자사 서비스 여부)로 추천율을 둔다. 기본 채점 방식이며 예외·감점이 따로 있다. 공고 항목에 관련 경력 근거 문장을 연결해 사람이 판정을 검토하도록 지원한다. SI 성격 공고는 추천에서 제외한다.
- 본인 지원 의사결정에 사용. 실행 이력별 통계(원본 → 통과 → NEW → 매치 70↑), 즐겨찾기, 지원 폴더 생성, HTML 리포트.